

Методы аксессоры

На практике широко принято делать все атрибуты класса и объекта защищенными или приватными, и предоставлять публичные методы для доступа к этим атрибутам и для их изменения. Так гарантируется, что класс или объект, владеющий этими атрибутами, будет держать под контролем все вносимые в них изменения.

Геттеры

Метод, который возвращает значение атрибута и при этом не изменяет его, называется **геттером**. Геттеры дают возможность программному коду, находящемуся за пределами класса, получать значения атрибутов безопасным способом, не подвергая эти атрибуты изменению программным кодом, находящимся вне метода.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name                # имя кошки

    def get_name(self):                  # геттер, используется для
получения имени
        return self._name

cat = Cat('Кемаль')

print(cat.get_name())
```

ВЫВОДИТ:

Кемаль

Сеттеры

Метод, который сохраняет значение в атрибуте либо изменяет значение атрибута, называется **сеттером**. Сеттеры могут управлять тем, как атрибуты объекта изменяются. Когда программный код, находящийся вне класса, должен изменить в объекте значение атрибута, он, как правило, вызывает сеттер и передает новое значение в качестве аргумента. Если это необходимо, то метод, прежде чем он присвоит значение атрибуту, может выполнить проверку этого значения.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        # сеттер, используется для
изменения имени
        if isinstance(name, str) and name.isalpha(): # проверка имени перед
заменой
            self._name = name
        else:
            raise ValueError('Некорректное имя')

cat = Cat('Кемаль')
print(cat.get_name())

cat.set_name('Роджер')
print(cat.get_name())
```

ВЫВОДИТ:

```
Кемаль
Роджер
```

В то время как код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

cat = Cat('Кемаль')

cat.set_name(-1) # некорректное имя
```

приводит к возбуждению исключения:

```
ValueError: Некорректное имя
```

Делитеры

Метод, который удаляет атрибут из объекта, называется **делитером**.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    def del_name(self):                    # делитер, используется для
удаления имени                          del self._name

cat = Cat('Кемаль')

cat.del_name()

print(cat.get_name())
```

приводит к возбуждению исключения `AttributeError`:

```
AttributeError: 'Cat' object has no attribute '_name'
```

Примечания

Примечание 1. Возбуждение исключения считается стандартной практикой при попытке установить атрибуту некорректное значение. Исключение может возбуждаться как при изменении значения атрибута, так и при создании экземпляра класса. В качестве примера рассмотрим тип `date` из модуля `datetime`.

Приведенный ниже код:

```
from datetime import date

today = date(2022, 20, 3)                # некорректное значение
месяца
```

приводит к возбуждению исключения:

ValueError: month must be in 1..12

Примечание 2. Названия геттер, сеттер и делитер происходят от английских getter, setter и deleter и используются напрямую без перевода.

Made with  by BEEGEEK

